

# ESP32 Wetter Station Teil 2

Anthony Timmel

19. Mai 2026

# Inhaltsverzeichnis

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>1 Grundlagen</b>                                  | <b>4</b>  |
| 1.1 Begriffe des MQTT-Protokolls                     | 4         |
| 1.1.1 MQTT-Broker                                    | 4         |
| 1.1.2 MQTT-Client                                    | 4         |
| 1.1.3 MQTT-Topic und dessen Grundregeln              | 4         |
| 1.1.4 Topic Wildcard einstufig und mehrstufig        | 4         |
| 1.1.5 Stufenmodell Quality of Service (QoS)          | 5         |
| 1.1.6 Last Will Testament                            | 6         |
| 1.2 Verbindungsmanagement zwischen Client und Server | 7         |
| 1.2.1 Retained Messages                              | 7         |
| 1.2.2 Persistent Sessions                            | 7         |
| 1.2.3 Keep Alive                                     | 7         |
| 1.2.4 Web Sockets Transport                          | 7         |
| 1.3 Pub/Sub Messaging                                | 7         |
| 1.4 MQTT-Client                                      | 8         |
| <b>2 Internetverbindung für den ESP32</b>            | <b>9</b>  |
| 2.1 Planung                                          | 9         |
| 2.2 Programmcode                                     | 9         |
| <b>3 MQTT-Client</b>                                 | <b>11</b> |
| 3.1 Planung                                          | 11        |
| 3.2 Programmcode                                     | 11        |
| <b>4 Verarbeitung der Daten in NODE-RED</b>          | <b>12</b> |
| 4.1 MQTT-Client innerhalb von NODE-RED               | 12        |
| 4.2 Temperatur                                       | 15        |
| 4.2.1 Vorbereitung der Daten für das UI              | 15        |
| 4.2.2 UI Template für aktuelle Daten                 | 15        |
| 4.2.3 UI Template für historie Daten                 | 17        |
| 4.3 Luftfeuchtigkeit                                 | 21        |
| 4.3.1 Vorbereitung der Daten für das UI              | 21        |
| 4.3.2 UI Template für aktuelle Daten                 | 21        |
| 4.3.3 UI Template für historie Daten                 | 23        |
| 4.4 Bewegung                                         | 27        |
| 4.4.1 Vorbereitung der Daten für das UI              | 27        |
| 4.4.2 UI Template für aktuelle Daten                 | 27        |
| 4.4.3 UI Template für historie Daten                 | 28        |
| 4.5 Helligkeit                                       | 32        |
| 4.5.1 Vorbereitung der Daten für das UI              | 32        |
| 4.5.2 UI Template für aktuelle Daten                 | 32        |

|          |                                          |           |
|----------|------------------------------------------|-----------|
| 4.5.3    | UI Template für historie Daten . . . . . | 35        |
| <b>5</b> | <b>Ganzer Programmcode für ESP32</b>     | <b>38</b> |

Im Rahmen des ersten Teils dieser Dokumentation setzen wir uns intensiv mit der initialen Aufgabenstellung aus dem Unterricht des Lernfeldes 7 (LF7) auseinander. Auf den nachfolgenden Seiten wird Ihnen das Projekt *Wetterstation ESP32* in seiner zweiten Phase detailliert und in schriftlicher Form präsentiert, um einen fundierten Überblick über die Zielsetzung und den Aufbau zu geben.

Diese Dokumentation ist der zweite Teil der Dokumentationsreihe, über den *ESP32* als Wetterstation.

# Kapitel 1

## Grundlagen

Innerhalb des zweiten Teils, kommen neue softwarebasierte Technologien zum Einsatz. Damit, bei der Referenzierung auf diese Begriffe & bei dem Ablauf der Beschreibung des Programmcodes keine Komplikationen in dem Verständniss des Ablaufs auftreten, werden die Begrifflichkeiten, im Umfang der zweiten Aufgabenstellung erklärt. Für eine detailliertere Beschreibung der einzelnen Begriffe, empfiehlt sich das Lesen der Beiträge auf [HiveMQ](#)

### 1.1 Begriffe des MQTT-Protokolls

In den folgenden Sektionen, werden Ihnen folgende Begriffe erklärt. Dies geschieht innerhalb des Rahmens von *Skript Teil 3*.

#### 1.1.1 MQTT-Broker

Der MQTT-Broker ist vergleichbar mit der Poststation. Der Broker erhält Nachrichten von den Clients & sendet diese an andere Clients weiter, die in diesen bestimmten Topics subscribed haben.

#### 1.1.2 MQTT-Client

Der MQTT-Client ist der Part, der innerhalb von z.B. IoT-Geräten eingebaut ist & für die Datenübertragung sowie Kommunikation verwendet wird. Der Client stellt eine Verbindung zum Broker her & subscribed zu ausgewählten Topics &/oder sendet ebenfalls auf diese Daten an den Broker zurück.

#### 1.1.3 MQTT-Topic und dessen Grundregeln

Eine MQTT-Topic ist eine Art Channel für die Kommunikation. Diese können basierend auf Pfaden von einander getrennt werden. Dabei ermöglicht es einem, nur auf bestimmte Aktivitäten zu reagieren, anstatt die Nachrichten erst Clientseitig zu verwalten. Dies reduziert zum einen die Menge an Daten die versendet werden, ebenfalls benötigt der Client weniger Funktionalität bei der Nachrichtenpriorisierung & Zuordnung, da er nur das Erhält, was er selbst gewählt hat.

#### 1.1.4 Topic Wildcard einstufig und mehrstufig

In MQTT gibt es zwei verschiedene Arten von Topic-Wildcards. Die erste Variante sind die einstufigen Wildcards, in MQTT wird hierfür das + benutzt. Eine Wildcard könnte daher wie folgt aussehen `abc/outlet01/+/audiQ3`. Die einstufige Wildcard würde nun alle Topics subscriben z.B. (`abc/outlet01/alert0/audiQ3`, `abc/outlet01/alert1/audiQ3` usw.).

Mehrstufige Topic Wildcards werden in MQTT mit dem # deklariert. Hierbei muss jedoch die Wildcard als letzter Pfad Parameter gesetzt sein. `abc/outlet01/alert0/#` wäre daher eine **valide** Wildcard. `abc/outlet01/#/audiQ3` ist hierbei eine **invalide** Wildcard

### 1.1.5 Stufenmodell Quality of Service (QoS)

Das Stufenmodell QoS ist für die Priorisierung von dem Datenverkehr im globalen & privaten Netzwerk zuständig. Hierbei gibt es folgende 4 Stufen der Priorisierung.

0-1 (Best Effort): Standarddatenverkehr (Web-Surfen, E-Mail).

2-3 (Best Effort+): Höhere Priorität als Standard.

4-5 (Echtzeit/Streaming): Reserviert für Video-Streaming, Telefonie (VoIP). Hier sind Verzögerungen (Jitter) und Paketverluste kritisch.

6-7 (Netzwerksteuerung): Höchste Priorität, reserviert für Netzwerk-Routing-Protokolle

### 1.1.6 Last Will Testament

Last Will Testament (LWT) ist für das Verwalten von Topics ausschlaggebend. Hierbei sendet der Broker bei einem unerwarteten disconnect von einem Client, eine Nachricht an alle anderen Clients in dieser Topic. In dem ersten Bild sehen Sie das Connect MQTT-Packet. Innerhalb dieses gibt es den Parameter

| MQTT-Packet:               |  |                   |
|----------------------------|--|-------------------|
| CONNECT                    |  |                   |
|                            |  |                   |
| contains:                  |  | Example           |
| clientId                   |  | "client-1"        |
| cleanSession               |  | true              |
| username (optional)        |  | "hans"            |
| password (optional)        |  | "letmein"         |
| lastWillTopic (optional)   |  | "/hans/will"      |
| lastWillQos (optional)     |  | 2                 |
| lastWillMessage (optional) |  | "unexpected exit" |
| lastWillRetain (optional)  |  | false             |
| keepAlive                  |  | 60                |

Abbildung 1.1: Last Will Message im Connect Packet (Grafik von HiveMQ)

| MQTT-Packet: |  |
|--------------|--|
| DISCONNECT   |  |
|              |  |
| no payload   |  |

Abbildung 1.2: Last Will Message beim Disconnect (Grafik von HiveMQ)

`lastWillMessage`, hier kann eine eigene Nachricht verfasst werden. Ebenfalls gibt es weitere Einstellungsmöglichkeiten für den LWT, wie z.B. in welcher Topic der LWT gesendet werden soll, oder in welcher Priorität *QoS* die Nachricht versendet werden soll.

## 1.2 Verbindungsmanagement zwischen Client und Server

In den folgenden Sektionen, werden Ihnen folgende Begriffe erklärt. Dies geschieht innerhalb des Rahmens von *Skript Teil 3*.

### 1.2.1 Retained Messages

Eine retained Message ist eine Nachricht, die auf dem Server (in unserem Fall Broker) gespeicherte Nachricht. Damit diese Nachricht gespeichert wird, muss die **retained**-Flag aktiv sein. Dann speichert der Broker vorübergehend die Nachricht für diese Topic inkl. QoS auf dem Server. Sobald sich ein neuer Client mit der Topic per Subscribe verbindet, erhält er diese Nachricht.

### 1.2.2 Persistent Sessions

Persistent Sessions haben Ähnlichkeiten zu den Retained Messages, erweitern jedoch den Funktionsrahmen um ein weiteres. Bei einer Persistent Session speichert der Server alle nicht zustellbaren Nachrichten an den Client in den subscribten Topics & versucht es erneut, sobald der MQTT-Client wieder mit dem Broker verbunden ist. Damit der Broker überhaupt erstmal bescheid weiß, dass der Client eine Persistent Session erstellen möchte, muss dies bei dem Verbinden mit dem Broker, übermittelt werden.

### 1.2.3 Keep Alive

Keep Alive ist eine Funktionalität um eine Verbindung mit dem Broker aufrecht zu erhalten, obwohl kein Nachrichtenaustausch zwischen Broker & Client stattfindet. Diese Funktionalität kann ebenfalls bei dem erstellen des Clients eingestellt werden.

### 1.2.4 Web Sockets Transport

Web Sockets Transport ist eine Möglichkeit das MQTT Protokoll zu erweitern & für die Browser möglich zu machen. Dabei wird das MQTT Protokoll mit dem WebSocket Protokoll erweitert, dadurch können Browser ebenfalls an einer Topic im MQTT-Broker subscriben & die Daten verwalten.

## 1.3 Pub/Sub Messaging

Pub/Sub Messaging ist eine Protokollfamilie. Hierbei werden Nachrichten in sogenannten **Channels** oder **Topics** versendet. Man könnte dies mit den verschiedenen Bereichen in einem Amt vergleichen, denn je nach Channel, geht die Nachricht an andere Empfänger über. Jedoch bleiben für den Clienten die Empfänger unbekannt.

Pub/Sub ist ein serverbasierter Nachrichtenprotokoll, d.h. die Nachricht, die ein Client versendet, werden an einen Server gesendet. Dieser *Verwaltet* die Eingehenden & Ausgehenden Verbindungen, den andere Clients können sich an diese Topics anklippen. Dadurch wird die Nachricht, sobald diese bei dem Server ankommt, automatisch an diese Clienten versendet. In dieser Grafik sieht man zwei verschiedene **Publisher** Clients. Diese senden eine Nachricht auf **/order** & **/inventory**. Beide senden die Nachricht mit dem Channel/Topic an den Pub/Sub Server. Hinter dem Pub/Sub Server befinden sich drei unterschiedliche Clients. Zwei dieser Clients hören auf Nachrichten innerhalb des **/order** Channels & einer auf den **/inventory**. Die Verbindung zwischen Client & Server bestehen aus WebSocket Verbindungen, damit der Server & der Client sich gegenseitig Nachrichten zusenden zu können.



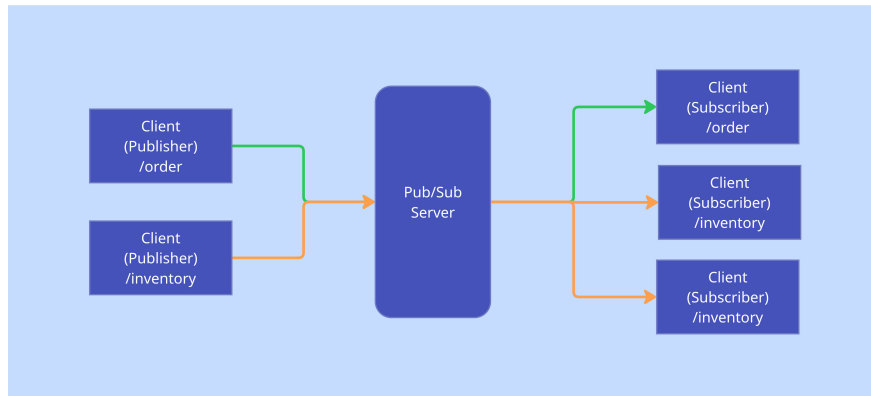


Abbildung 1.3: Grafische Darstellung des Pub/Sub Verfahrens

## 1.4 MQTT-Client

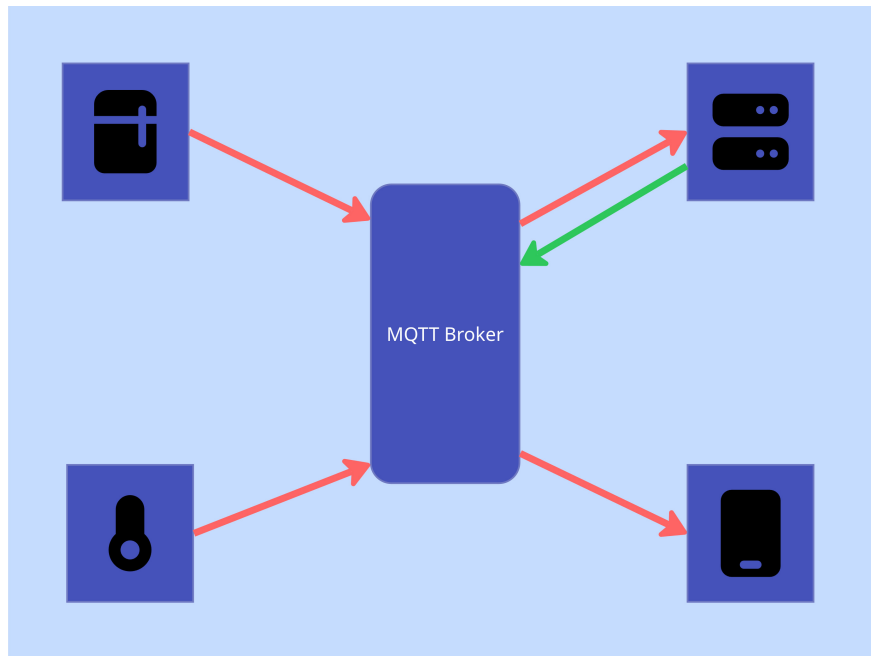


Abbildung 1.4: Grafik von MQTT

Diese Grafik verbildlicht die Kommunikation zwischen den einzelnen Geräten wie den Kühlschrank, Thermometer, Server & einem Handy. Hierbei werden die Daten mithilfe von dem MQTT-Client an den Broker gesendet. Der Broker sendet diese Daten dann an alle Clients, die den **Channel** subscribed haben.

Innerhalb dieses zweiten Teils werden Daten an einen externen Dienst versendet. Dafür gibt es verschiedene Wege, diesen Prozess zu absolvieren.

Das MQTT-Protokoll ist ein klassisches Pub/Sub Modell in der IoT Welt. Hierbei agiert es als leichtgewichtiges Messaging Protokoll für die Übermittlung von Daten, von unseren Client an den Server.

## Kapitel 2

# Internetverbindung für den ESP32

Damit Daten überhaupt von dem ESP32 an den MQTT-Broker übermittelt werden können, benötigt der ESP32 eine Internetverbindung.

Für die technische Umsetzung wurde sich auf die [ESP32 WLAN Modul Dokumentation](#) bezogen.

### 2.1 Planung

Der ESP32 soll sich mit einem W-LAN AccessPoint in der Nähe verbinden & dadurch eine internetfähige Verbindung aufbauen, damit später, eine Möglichkeit besteht, den MQTT-Broker zu erreichen.

### 2.2 Programmcode

```
1  import network
2
3  wlan = network.WLAN(network.STA_IF)
4
5  def do_connect():
6      wlan.active(True)
7      if not wlan.isconnected():
8          print('connecting to network...')
9          wlan.connect('FI24-Hotspot', 'BszWsw11#')
10         while not wlan.isconnected():
11             print("WLAN is not connected")
12             pass
13
14     print('network config:', wlan.ifconfig())
15
16 do_connect()
17
```

Zuerst importieren wir das Modul **network**. In Zeile 3 deklarieren wir das WLAN-Modul des ESP32, indem wir aus dem Netzwerk-Modul die Klasse **WLAN** instanziiieren. **network.STA\_IF** bedeutet hierbei, das wir es als Station Interface deklarieren, also ein stationärer Zugriffspunkt (Access Point).

Damit wir später im Programmcode einfacher eine W-LAN Verbindung herstellen können, deklarieren wir die Funktion `do_connect()`. Innerhalb des Funktionsbodies, aktivieren wir zuerst die WLAN-Funktionalität, damit wir uns später mit einem Access Point verbinden können. Wir überprüfen zuerst, ob wir mit einem Access-Point verbunden sind. Dies verhindert, dass wir die Verbindung zu dem Access-Point neu aufbauen, obwohl wir bereits eine Verbindung haben.

In Zeile 9 verbinden wir uns mit dem definierten Access Point & überprüfen in der while-Schleife dann so lange, ob die Verbindung erfolgreich war. Erst nach einer erfolgreichen Verbindung lassen wir die while-Schleife los & geben die IP-Informationen in die Konsole aus.

Damit die WLAN Verbindung initialisiert werden kann, wird die Funktion `do_connect()` aufgerufen.

# Kapitel 3

## MQTT-Client

### 3.1 Planung

Der ESP32 soll bei einer internetfähigen Verbindung, sich mit dem MQTT-Broker verbinden & pro Durchlauf die Daten an den Broker senden, über eine zuvor definierte Topic.

### 3.2 Programmcode

```
1  from umqtt.simple import MQTTClient
2
3  client_id = "ESP_32_MQTT"
4  server = "mosquitto.nodered-fi.ipv64.net"
5  port = 0
6  user = "FI"
7  password = "FI"
8  keepalive = 1
9  mqtt_client = MQTTClient(client_id, server, port, user, password, keepalive=0,
10                           ssl=False, ssl_params={})
11
12 def mqtt_connect():
13     mqtt_client.connect()
14
15 def send_mqtt_message(message):
16     channel = "Met/FI/Timmel"
17     mqtt_client.publish(channel, message)
```

## Kapitel 4

# Verarbeitung der Daten in NODE-RED

### 4.1 MQTT-Client innerhalb von NODE-RED

Alle Daten kommen zuerst über einen MQTT-Client in Node-RED. Dieser Client ist an die selbe Topic subscribed, wie unser MQTT-Client für den ESP32 die Daten an den Broker sendet.

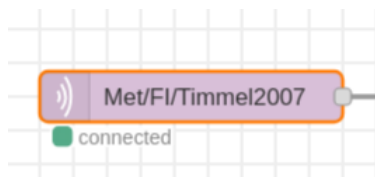


Abbildung 4.1:

Die Node *mqtt in* representiert einen MQTT Client, mitdem wir in diesem Beispiel eine Topic subscriben & die Daten anschließend innerhalb von Node-RED weiterverarbeiten.

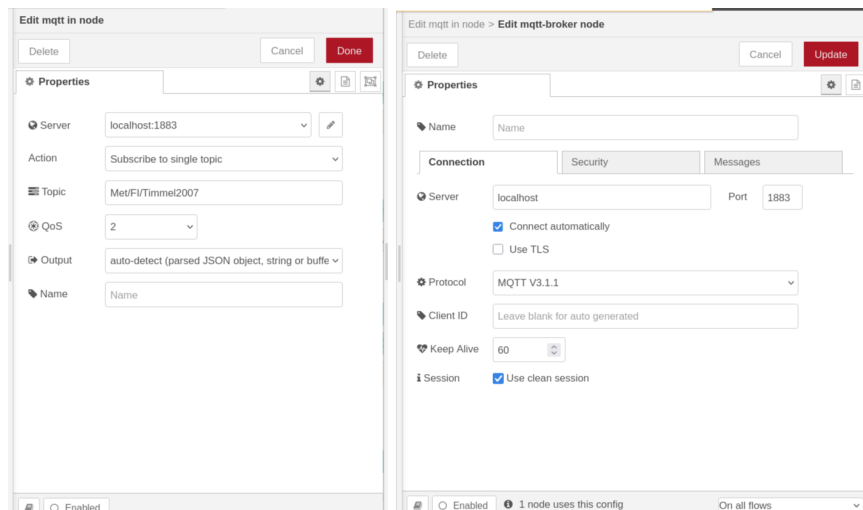


Abbildung 4.2:

Bevor die Node jedoch die Daten empfangen kann, müssen wir diese zuvor Einrichten. Hierfür setzen wir die richtige Serverkonfiguration ein, in unserem Fall die IP 127.0.0.1 mit dem dazugehörigen Port 1883. Ebenfalls müssen wir die Zugangsdaten für den MQTT-Broker übergeben, weshalb wir diese angeben

müssen. Nachdem wir die Serververbindung zum Broker eingerichtet haben, setzen wir die passende Topic zum subscriben. Bestätigen die Änderungen und deployen den Flow.

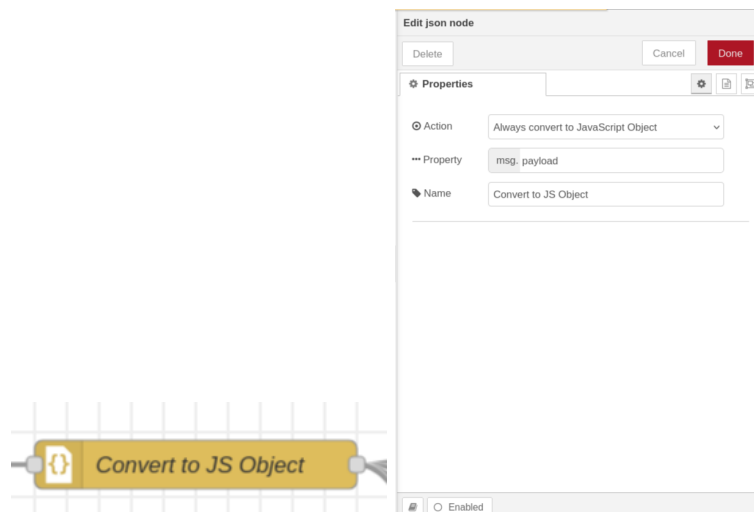


Abbildung 4.3:

Mit der **Convert to JSON-Object** Node, konvertieren wir den JSON-String in ein valides JSON-Objekt.

## 4.2 Temperatur

### 4.2.1 Vorbereitung der Daten für das UI



Abbildung 4.4:

Bevor wir die Daten für die Temperatur in die Visualisierung leiten können, müssen wir zuvor die richtigen Daten zuweisen.

```
1 msg.payload = msg.payload.Temperatur;
2
3 return msg;
4
```

Mithilfe dieses JavaScript Codes setzen wir die `msg.payload` Variable auf den Temperatur Wert, den wir zuvor erhalten haben.

### 4.2.2 UI Template für aktuelle Daten

```
1 <div class="temp-widget" ng-class="getTempClass(msg.payload)">
2   <div class="glass-overlay"></div>
3   <div class="content">
4     <span class="label">Temperatur</span>
5     <div class="temp-value">
6       <span class="number">{{msg.payload | number:1}}</span><span class="
unit">°C</span>
7     </div>
8     <div class="status-indicator">
9       <div class="pulse-dot"></div>
10      <span class="status-text">{{getStatusText(msg.payload)}}</span>
11    </div>
12  </div>
13 </div>
14
15 <style>
16
17   .temp-widget {
18     position: relative;
19     overflow: hidden;
20     border-radius: 18px;
21     padding: 20px;
22     color: white;
23     font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
24     min-height: 120px;
25     transition: background 0.8s ease;
26   }
27
28   .glass-overlay {
29     position: absolute;
30     top: 0;
31     left: 0;
```



```

32     right: 0;
33     bottom: 0;
34     background: linear-gradient(135deg, rgba(255, 255, 255, 0.1) 0%, rgba(255,
255, 255, 0) 100%);
35     z-index: 1;
36 }
37
38 .content {
39     position: relative;
40     z-index: 2;
41 }
42
43 .label {
44     font-size: 0.85rem;
45     text-transform: uppercase;
46     letter-spacing: 1px;
47     opacity: 0.8;
48 }
49
50 .temp-value {
51     margin: 10px 0;
52     display: flex;
53     align-items: baseline;
54 }
55
56 .number {
57     font-size: 3.5rem;
58     font-weight: 800;
59     line-height: 1;
60 }
61
62 .unit {
63     font-size: 1.5rem;
64     margin-left: 5px;
65     font-weight: 300;
66 }
67
68 .status-indicator {
69     display: flex;
70     align-items: center;
71     gap: 8px;
72     font-size: 0.9rem;
73 }
74
75 .pulse-dot {
76     width: 8px;
77     height: 8px;
78     background-color: white;
79     border-radius: 50%;
80     box-shadow: 0 0 0 0 rgba(255, 255, 255, 0.7);
81     animation: pulse 2s infinite;
82 }
83
84 @keyframes pulse {
85     0% {
86         transform: scale(0.95);
87         box-shadow: 0 0 0 0 rgba(255, 255, 255, 0.7);

```

```

88     }
89     70% {
90         transform: scale(1);
91         box-shadow: 0 0 0 10px rgba(255, 255, 255, 0);
92     }
93     100% {
94         transform: scale(0.95);
95         box-shadow: 0 0 0 0 rgba(255, 255, 255, 0);
96     }
97 }
98
99 .temp-cold {
100     background: linear-gradient(135deg, #1bd5ed 0%, #15a2b8 100%);
101 }
102
103 .temp-normal {
104     background: linear-gradient(135deg, #88f411 0%, #72cc0e 100%);
105 }
106
107 .temp-warm {
108     background: linear-gradient(135deg, #d99f0e 0%, #b8860c 100%);
109 }
110
111 .temp-hot {
112     background: linear-gradient(135deg, #d90e0e 0%, #a30a0a 100%);
113 }
114
115 </style>
116
117 <script>
118     (function (scope) {
119         scope.getTempClass = function (temp) {
120             if (temp > 25) return 'temp-hot';
121             if (temp > 20) return 'temp-warm';
122             if (temp > 15) return 'temp-normal';
123             return 'temp-cold';
124         };
125
126         scope.getStatusText = function (temp) {
127             if (temp > 25) return 'Sehr warm';
128             if (temp > 20) return 'Angenehm';
129             if (temp > 15) return 'Kühl';
130             return 'Kalt';
131         };
132     })(scope);
133 </script>

```

## 4.2.3 UI Template für historie Daten

```

1 let history = global.get('temp-history') || [];
2
3 let timeLabel = new Date().toLocaleTimeString('de-DE', {
4     hour: '2-digit',
5     minute: '2-digit',
6     second: '2-digit'
7 });
8

```

```

9  history.push({
10      time: timeLabel,
11      value: msg.payload,
12      timestamp: new Date()
13  });
14
15  let hourAgo = Date.now() - (60 * 60 * 1000);
16  history = history.filter(record => record.timestamp >= hourAgo);
17
18  global.set('temp-history', history);
19
20  msg.payload = history;
21  return msg;

```

```

1  <script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/3.9.1/chart.min.js"><
    /script>
2
3  <div class="history-card">
4      <div class="chart-header">
5          <span class="chart-title">Verlauf (24h)</span>
6          <span class="chart-avg">Schnitt: {{avgTemp}}°C</span>
7      </div>
8      <div class="chart-container">
9          <canvas id="tempHistoryChart"></canvas>
10     </div>
11 </div>
12
13 <style>
14     .history-card {
15         background: #e3e3e3;
16         border-radius: 16px;
17         padding: 16px 16px 8px 16px;
18         color: white;
19         font-family: 'Segoe UI', sans-serif;
20         height: calc(100% - 32px);
21         display: flex;
22         flex-direction: column;
23     }
24
25     .chart-header {
26         display: flex;
27         justify-content: space-between;
28         align-items: center;
29         margin-bottom: 15px;
30     }
31
32     .chart-title {
33         font-size: 0.9rem;
34         font-weight: 600;
35         text-transform: uppercase;
36         letter-spacing: 1px;
37         color: #a0aec0;
38     }
39
40     .chart-avg {
41         font-size: 0.85rem;
42         background: rgba(255, 255, 255, 0.1);

```

```

43     padding: 2px 8px;
44     border-radius: 4px;
45     color: #a0aec0;
46 }
47
48 .chart-container {
49     flex-grow: 1;
50     position: relative;
51 }
52 </style>
53
54 <script>
55     (function (scope) {
56         let chart;
57
58         function initChart(data) {
59             const ctx = document.getElementById('tempHistoryChart').getContext('2d
60
61             const gradient = ctx.createLinearGradient(0, 0, 0, 200);
62             gradient.addColorStop(0, 'rgba(27, 213, 237, 0.4)');
63             gradient.addColorStop(1, 'rgba(27, 213, 237, 0)');
64
65             chart = new Chart(ctx, {
66                 type: 'line',
67                 data: {
68                     labels: data.map(d => d.time),
69                     datasets: [{
70                         label: 'Temperatur',
71                         data: data.map(d => d.value),
72                         borderColor: '#1bd5ed',
73                         borderWidth: 3,
74                         pointRadius: 0,
75                         pointHoverRadius: 5,
76                         fill: true,
77                         backgroundColor: gradient,
78                         tension: 0.4
79                     }]
80                 },
81                 options: {
82                     responsive: true,
83                     maintainAspectRatio: false,
84                     plugins: {
85                         legend: {display: false}
86                     },
87                     layout: {
88                         padding: {
89                             bottom: 10
90                         }
91                     },
92                     scales: {
93                         x: {
94                             display: true,
95                             grid: {
96                                 display: false,
97                                 drawBorder: false
98                             }

```

```

99         ticks: {
100             color: '#718096',
101             font: {size: 10},
102             maxRotation: 0,
103             autoSkip: true,
104             maxTicksLimit: 8
105         }
106     },
107     y: {
108         grid: {
109             color: 'rgba(255,255,255,0.05)',
110             drawBorder: false
111         },
112         ticks: {
113             color: '#718096',
114             font: {size: 10}
115         }
116     }
117 }
118 }
119 });
120 }
121
122 scope.$watch('msg', function (msg) {
123     if (!msg || !msg.payload) return;
124
125     if (Array.isArray(msg.payload)) {
126         const values = msg.payload.map(d => d.value);
127         scope.avgTemp = (values.reduce((a, b) => a + b, 0) / values.length
128         ).toFixed(1);
129
130         if (!chart) {
131             initChart(msg.payload);
132         } else {
133             chart.data.labels = msg.payload.map(d => d.time);
134             chart.data.datasets[0].data = values;
135             chart.update('none');
136         }
137     });
138 })(scope);
139 </script>

```

## 4.3 Luftfeuchtigkeit

### 4.3.1 Vorbereitung der Daten für das UI



Abbildung 4.5:

Bevor wir die Daten für die Luftfeuchtigkeit in die Visualisierung leiten können, müssen wir zuvor die richtigen Daten zuweisen.

```
1 msg.payload = msg.payload.Luftfeuchtigkeit;
2
3 return msg;
4
```

Mithilfe dieses JavaScript Codes setzen wir die `msg.payload` Variable auf den Luftfeuchtigkeit's Wert, den wir zuvor erhalten haben.

### 4.3.2 UI Template für aktuelle Daten

```
1 <div class="hum-widget" ng-class="getHumClass(msg.payload)">
2   <div class="glass-overlay"></div>
3   <div class="content">
4     <span class="label">Luftfeuchtigkeit</span>
5     <div class="hum-value">
6       <span class="number">{{msg.payload | number:0}}</span><span class="
unit">%</span>
7     </div>
8     <div class="status-indicator">
9       <div class="pulse-dot"></div>
10      <span class="status-text">{{getHumStatus(msg.payload)}}</span>
11    </div>
12  </div>
13 </div>
14
15 <style>
16
17   .hum-widget {
18     position: relative;
19     overflow: hidden;
20     border-radius: 18px;
21     padding: 20px;
22     color: white;
23     font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
24     min-height: 120px;
25     transition: background 0.8s ease;
26   }
27
28   .glass-overlay {
29     position: absolute;
30     top: 0;
31     left: 0;
```

```

32     right: 0;
33     bottom: 0;
34     background: linear-gradient(135deg, rgba(255, 255, 255, 0.15) 0%, rgba
(255, 255, 255, 0) 100%);
35     z-index: 1;
36     pointer-events: none;
37 }
38
39 .content {
40     position: relative;
41     z-index: 2;
42 }
43
44 .label {
45     font-size: 0.85rem;
46     text-transform: uppercase;
47     letter-spacing: 1px;
48     opacity: 0.8;
49 }
50
51 .hum-value {
52     display: flex;
53     align-items: baseline;
54     margin-bottom: 5px;
55 }
56
57 .number {
58     font-size: 3.5rem;
59     font-weight: 800;
60     line-height: 1;
61 }
62
63 .unit {
64     font-size: 1.5rem;
65     margin-left: 5px;
66     font-weight: 300;
67 }
68
69 .status-indicator {
70     display: flex;
71     align-items: center;
72     gap: 8px;
73     font-size: 0.9rem;
74 }
75
76 .pulse-dot {
77     width: 8px;
78     height: 8px;
79     background-color: white;
80     border-radius: 50%;
81     animation: pulse 2s infinite;
82 }
83
84 @keyframes pulse {
85     0% {
86         transform: scale(0.95);
87         box-shadow: 0 0 0 0 rgba(255, 255, 255, 0.7);

```

```

88     }
89     70% {
90         transform: scale(1);
91         box-shadow: 0 0 0 10px rgba(255, 255, 255, 0);
92     }
93     100% {
94         transform: scale(0.95);
95         box-shadow: 0 0 0 0 rgba(255, 255, 255, 0);
96     }
97 }
98
99 .hum-dry {
100     background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
101 }
102
103 .hum-ideal {
104     background: linear-gradient(135deg, #b490ca 0%, #5ee7df 100%);
105 }
106
107 .hum-ok {
108     background: linear-gradient(135deg, #84fab0 0%, #8fd3f4 100%);
109 }
110
111 .hum-wet {
112     background: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
113 }
114
115 .hum-verywet {
116     background: linear-gradient(135deg, #1e3c72 0%, #2a5298 100%);
117 }
118 </style>
119
120 <script>
121     (function (scope) {
122         scope.getHumClass = function (hum) {
123             if (hum < 30) return 'hum-dry';
124             if (hum >= 30 && hum <= 45) return 'hum-ok';
125             if (hum > 45 && hum <= 60) return 'hum-ideal';
126             if (hum > 60 && hum <= 75) return 'hum-wet';
127             return 'hum-verywet';
128         };
129
130         scope.getHumStatus = function (hum) {
131             if (hum < 30) return 'Zu Trocken';
132             if (hum >= 30 && hum <= 45) return 'Leicht Trocken';
133             if (hum > 45 && hum <= 60) return 'Optimal';
134             if (hum > 60 && hum <= 75) return 'Feucht';
135             return 'Sehr Nass / Schimmelgefahr';
136         };
137     })(scope);
138 </script>

```

### 4.3.3 UI Template für historie Daten

```

1 let history = global.get('humid-history') || [];
2
3 let timeLabel = new Date().toLocaleTimeString('de-DE', {

```



```

4     hour: '2-digit',
5     minute: '2-digit',
6     second: '2-digit'
7 });
8
9 history.push({
10     time: timeLabel,
11     value: msg.payload,
12     timestamp: new Date()
13 });
14
15 let hourAgo = Date.now() - (60 * 60 * 1000);
16 history = history.filter(record => record.timestamp >= hourAgo);
17
18 global.set('humid-history', history);
19
20 msg.payload = history;
21 return msg;

```

```

1 <script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/3.9.1/chart.min.js"><
  /script>
2
3 <div class="history-card">
4     <div class="chart-header">
5         <span class="chart-title">Verlauf (24h)</span>
6         <span class="chart-avg">Schnitt: {{avgHumid}}%</span>
7     </div>
8     <div class="chart-container">
9         <canvas id="humidHistoryChart"></canvas>
10    </div>
11 </div>
12
13 <style>
14     .history-card {
15         background: #e3e3e3;
16         border-radius: 16px;
17         padding: 16px 16px 8px 16px;
18         color: white;
19         font-family: 'Segoe UI', sans-serif;
20         height: calc(100% - 32px);
21         display: flex;
22         flex-direction: column;
23     }
24
25     .chart-header {
26         display: flex;
27         justify-content: space-between;
28         align-items: center;
29         margin-bottom: 15px;
30     }
31
32     .chart-title {
33         font-size: 0.9rem;
34         font-weight: 600;
35         text-transform: uppercase;
36         letter-spacing: 1px;
37         color: #a0aec0;

```

```

38     }
39
40     .chart-avg {
41         font-size: 0.85rem;
42         background: rgba(255, 255, 255, 0.1);
43         padding: 2px 8px;
44         border-radius: 4px;
45         color: #a0aec0;
46     }
47
48     .chart-container {
49         flex-grow: 1;
50         position: relative;
51     }
52 </style>
53
54 <script>
55     (function (scope) {
56         let chart;
57
58         function initChart(data) {
59             const ctx = document.getElementById('humidHistoryChart').getContext('2
60 d');
61
62             const gradient = ctx.createLinearGradient(0, 0, 0, 200);
63             gradient.addColorStop(0, 'rgba(27, 213, 237, 0.4)');
64             gradient.addColorStop(1, 'rgba(27, 213, 237, 0)');
65
66             chart = new Chart(ctx, {
67                 type: 'line',
68                 data: {
69                     labels: data.map(d => d.time),
70                     datasets: [{
71                         label: 'Luftfeuchtigkeit',
72                         data: data.map(d => d.value),
73                         borderColor: '#1bd5ed',
74                         borderWidth: 3,
75                         pointRadius: 0,
76                         pointHoverRadius: 5,
77                         fill: true,
78                         backgroundColor: gradient,
79                         tension: 0.4
80                     }]
81                 },
82                 options: {
83                     responsive: true,
84                     maintainAspectRatio: false,
85                     plugins: {
86                         legend: {display: false}
87                     },
88                     layout: {
89                         padding: {
90                             bottom: 10
91                         }
92                     },
93                     scales: {
94                         x: {

```

```

94         display: true,
95         grid: {
96             display: false,
97             drawBorder: false
98         },
99         ticks: {
100             color: '#718096',
101             font: {size: 10},
102             maxRotation: 0,
103             autoSkip: true, maxTicksLimit: 8
104         }
105     },
106     y: {
107         grid: {
108             color: 'rgba(255,255,255,0.05)',
109             drawBorder: false
110         },
111         ticks: {
112             color: '#718096',
113             font: {size: 10}
114         }
115     }
116 }
117 }
118 });
119 }
120
121 scope.$watch('msg', function (msg) {
122     if (!msg || !msg.payload) return;
123
124     if (Array.isArray(msg.payload)) {
125         const values = msg.payload.map(d => d.value);
126         scope.avgHumid = (values.reduce((a, b) => a + b, 0) / values.
length).toFixed(1);
127
128         if (!chart) {
129             initChart(msg.payload);
130         } else {
131             chart.data.labels = msg.payload.map(d => d.time);
132             chart.data.datasets[0].data = values;
133             chart.update('none');
134         }
135     }
136 });
137 })(scope);
138 </script>

```

## 4.4 Bewegung

### 4.4.1 Vorbereitung der Daten für das UI



Abbildung 4.6:

Bevor wir die Daten für die Bewegung in die Visualisierung leiten können, müssen wir zuvor die richtigen Daten zuweisen.

```
1 msg.payload = msg.payload.Luftfeuchtigkeit;
2
3 return msg;
4
```

Mithilfe dieses JavaScript Codes setzen wir die `msg.payload` Variable auf den Bewegung Wert, den wir zuvor erhalten haben.

### 4.4.2 UI Template für aktuelle Daten

```
1 <style>
2   .motion-container {
3     display: flex;
4     flex-direction: column;
5     align-items: center;
6     justify-content: center;
7     padding: 20px;
8     font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
9     overflow-y: hidden;
10  }
11
12  .status-indicator-motion {
13    width: 80px;
14    height: 80px;
15    border-radius: 50%;
16    transition: all 0.5s ease;
17    display: flex;
18    align-items: center;
19    justify-content: center;
20    margin-bottom: 15px;
21    border: 4px solid #333;
22  }
23
24  .status-still {
25    background-color: #2c3e50;
26    box-shadow: 0 0 0px rgba(0, 0, 0, 0);
27  }
28
29  .status-active {
30    background-color: #e74c3c;
31    box-shadow: 0 0 20px #e74c3c;
32    animation: pulse-red 1.5s infinite;
```

```

33   }
34
35   .status-text {
36     font-size: 1.2em;
37     font-weight: bold;
38     text-transform: uppercase;
39     letter-spacing: 1px;
40   }
41
42   @keyframes pulse-red {
43     0% {
44       transform: scale(0.95);
45       box-shadow: 0 0 0 0 rgba(231, 76, 60, 0.7);
46     }
47
48     70% {
49       transform: scale(1);
50       box-shadow: 0 0 0 15px rgba(231, 76, 60, 0);
51     }
52
53     100% {
54       transform: scale(0.95);
55       box-shadow: 0 0 0 0 rgba(231, 76, 60, 0);
56     }
57   }
58 </style>
59
60 <div class="motion-container">
61   <div class="status-indicator-motion" ng-class="msg.payload ? 'status-active' :
62     'status-still'">
63     <i class="fa fa-user-secret" style="font-size: 30px; color: white;"></i>
64   </div>
65   <div class="status-text" ng-style="{ 'color': msg.payload ? '#e74c3c' : '#
66     bdc3c7' }">
67     {{msg.payload ? 'Bewegung erkannt' : 'Keine Bewegung'}}
68   </div>
69 </div>

```

#### 4.4.3 UI Template für historie Daten

```

1 const maxLogs = 10;
2
3 let log = context.get('motion-eventLog') || [];
4
5 if (msg.payload === true || msg.payload === 1 || msg.payload === "on") {
6
7   let now = new Date();
8   let timeString = now.toLocaleTimeString('de-DE', {
9     day: '2-digit',
10    month: '2-digit',
11    hour: '2-digit',
12    minute: '2-digit'
13  });
14
15  if (log.filter((element) => element.time === timeString).length === 0) {
16    log.unshift({
17      time: timeString,

```

```

18         id: Date.now()
19     });
20
21     if (log.length > maxLogs) {
22         log = log.slice(0, maxLogs);
23     }
24 }
25
26 context.set('motion-eventLog', log);
27 }
28
29 msg.payload = log;
30 return msg;

```

```

1 <div class="log-card">
2   <div class="log-header">
3     <span class="log-title">Ereignisprotokoll</span>
4     <span class="log-count">{{msg.payload.length}} Events</span>
5   </div>
6   <div class="log-container">
7     <div class="log-entry" ng-repeat="event in msg.payload | orderBy:'-$index'
8   ">
9       <div class="event-icon">
10         <i class="fa fa-walking"></i>
11       </div>
12       <div class="event-details">
13         <span class="event-name">Bewegung erkannt</span>
14         <span class="event-time">{{event.time}}</span>
15       </div>
16       <div class="event-indicator"></div>
17     </div>
18     <div class="log-empty" ng-if="!msg.payload || msg.payload.length === 0">
19       Keine Bewegungen registriert
20     </div>
21   </div>
22 </div>
23 <style>
24   .log-card {
25     background: #1a2634;
26     border-radius: 16px;
27     padding: 16px;
28     color: white;
29     font-family: 'Segoe UI', sans-serif;
30     height: calc(100% - 32px);
31     display: flex;
32     flex-direction: column;
33     box-shadow: 0 4px 15px rgba(0,0,0,0.3);
34   }
35
36   .log-header {
37     display: flex;
38     justify-content: space-between;
39     align-items: center;
40     margin-bottom: 12px;
41     padding-bottom: 8px;
42     border-bottom: 1px solid rgba(255,255,255,0.1);

```

```

43 }
44
45 .log-title {
46     font-size: 0.85rem;
47     font-weight: 600;
48     text-transform: uppercase;
49     color: #a0aec0;
50 }
51
52 .log-count {
53     font-size: 0.75rem;
54     background: #2d3748;
55     padding: 2px 8px;
56     border-radius: 10px;
57     color: #63b3ed;
58 }
59
60 .log-container {
61     flex-grow: 1;
62     overflow-y: auto;
63     padding-right: 5px;
64 }
65
66 .log-container::-webkit-scrollbar { width: 4px; }
67 .log-container::-webkit-scrollbar-thumb { background: #4a5568; border-radius:
10px; }
68
69 .log-entry {
70     display: flex;
71     align-items: center;
72     padding: 10px;
73     margin-bottom: 8px;
74     background: rgba(255,255,255,0.03);
75     border-radius: 10px;
76     transition: transform 0.2s;
77     border-left: 3px solid #63b3ed;
78 }
79
80 .event-icon {
81     width: 32px;
82     height: 32px;
83     background: rgba(99, 179, 237, 0.1);
84     border-radius: 8px;
85     display: flex;
86     align-items: center;
87     justify-content: center;
88     margin-right: 12px;
89     color: #63b3ed;
90 }
91
92 .event-details {
93     display: flex;
94     flex-direction: column;
95     flex-grow: 1;
96 }
97
98 .event-name {

```

```
99         font-size: 0.85rem;
100         font-weight: 500;
101     }
102
103     .event-time {
104         font-size: 0.75rem;
105         color: #718096;
106     }
107
108     .event-indicator {
109         width: 8px;
110         height: 8px;
111         background: #63b3ed;
112         border-radius: 50%;
113         opacity: 0.6;
114     }
115
116     .log-empty {
117         text-align: center;
118         padding: 20px;
119         color: #718096;
120         font-style: italic;
121     }
122 </style>
```



## 4.5 Helligkeit

### 4.5.1 Vorbereitung der Daten für das UI



Abbildung 4.7:

Bevor wir die Daten für die Helligkeit in die Visualisierung leiten können, müssen wir zuvor die richtigen Daten zuweisen.

```
1 msg.payload = msg.payload.Helligkeit;
2
3 return msg;
4
```

Mithilfe dieses JavaScript Codes setzen wir die `msg.payload` Variable auf den Helligkeit Wert, den wir zuvor erhalten haben.

### 4.5.2 UI Template für aktuelle Daten

```
1 <div class="lum-widget" ng-class="getLumClass(msg.payload)">
2   <div class="glass-overlay"></div>
3   <div class="content">
4     <span class="label">Helligkeit</span>
5     <div class="lum-value">
6       <span class="number">{{msg.payload | number:0}}</span><span class="
unit">Lumen</span>
7     </div>
8     <div class="status-indicator">
9       <div class="pulse-dot"></div>
10      <span class="status-text">{{getLumStatus(msg.payload)}}</span>
11    </div>
12  </div>
13 </div>
14
15 <style>
16
17   .lum-widget {
18     position: relative;
19     overflow: hidden;
20     border-radius: 18px;
21     padding: 20px;
22     color: white;
23     font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
24     min-height: 120px;
25     transition: background 0.8s ease;
26   }
27
28   .glass-overlay {
29     position: absolute;
30     top: 0;
31     left: 0;
```

```

32     right: 0;
33     bottom: 0;
34     background: linear-gradient(135deg, rgba(255, 255, 255, 0.15) 0%, rgba
(255, 255, 255, 0) 100%);
35     z-index: 1;
36     pointer-events: none;
37 }
38
39 .content {
40     position: relative;
41     z-index: 2;
42 }
43
44 .label {
45     font-size: 0.85rem;
46     text-transform: uppercase;
47     letter-spacing: 1px;
48     opacity: 0.8;
49 }
50
51 .lum-value {
52     display: flex;
53     align-items: baseline;
54     margin-bottom: 5px;
55 }
56
57 .number {
58     font-size: 3.5rem;
59     font-weight: 800;
60     line-height: 1;
61 }
62
63 .unit {
64     font-size: 1.5rem;
65     margin-left: 5px;
66     font-weight: 300;
67 }
68
69 .status-indicator {
70     display: flex;
71     align-items: center;
72     gap: 8px;
73     font-size: 0.9rem;
74 }
75
76 .pulse-dot {
77     width: 8px;
78     height: 8px;
79     background-color: white;
80     border-radius: 50%;
81     animation: pulse 2s infinite;
82 }
83
84 @keyframes pulse {
85     0% {
86         transform: scale(0.95);
87         box-shadow: 0 0 0 0 rgba(255, 255, 255, 0.7);

```

```

88     }
89     70% {
90         transform: scale(1);
91         box-shadow: 0 0 0 10px rgba(255, 255, 255, 0);
92     }
93     100% {
94         transform: scale(0.95);
95         box-shadow: 0 0 0 0 rgba(255, 255, 255, 0);
96     }
97 }
98
99 .lum-night {
100     background: linear-gradient(135deg, #141e30 0%, #243b55 100%);
101 }
102
103 .lum-dim {
104     background: linear-gradient(135deg, #2c3e50 0%, #4ca1af 100%);
105 }
106
107 .lum-indoor {
108     background: linear-gradient(135deg, #f12711 0%, #f5af19 100%);
109 }
110
111 .lum-bright {
112     background: linear-gradient(135deg, #fffb5 0%, #b20a2c 100%);
113 }
114
115 .lum-sunny {
116     background: linear-gradient(135deg, #ffce00 0%, #ff8c00 100%);
117 }
118
119 .lum-max {
120     background: linear-gradient(135deg, #ffffff 0%, #ffefba 100%);
121     color: #333 !important;
122 }
123 </style>
124
125 <script>
126     (function (scope) {
127         scope.getLumClass = function (lum) {
128             if (lum < 10) return 'lum-night';
129             if (lum >= 10 && lum <= 100) return 'lum-dim';
130             if (lum > 100 && lum <= 500) return 'lum-indoor';
131             if (lum > 500 && lum <= 2000) return 'lum-bright';
132             return 'lum-max';
133         };
134
135         scope.getLumStatus = function (lum) {
136             if (lum < 10) return 'Nacht';
137             if (lum >= 10 && lum <= 100) return 'Dämmerung';
138             if (lum > 100 && lum <= 500) return 'Wohnlicht';
139             if (lum > 500 && lum <= 2000) return 'Tageslicht';
140             return 'Direketes Licht';
141         };
142     })(scope);
143 </script>

```

### 4.5.3 UI Template für historie Daten

```
1 let history = global.get('luminance-history') || [];
2
3 let timeLabel = new Date().toLocaleTimeString('de-DE', {
4   hour: '2-digit',
5   minute: '2-digit',
6   second: '2-digit'
7 });
8
9 history.push({
10   time: timeLabel,
11   value: msg.payload,
12   timestamp: new Date()
13 });
14
15 let hourAgo = Date.now() - (60 * 60 * 1000);
16 history = history.filter(record => record.timestamp >= hourAgo);
17
18 global.set('luminance-history', history);
19
20 msg.payload = history;
21 return msg;
```

```
1 <script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/3.9.1/chart.min.js"><
  /script>
2
3 <div class="history-card">
4   <div class="chart-header">
5     <span class="chart-title">Verlauf (24h)</span>
6     <span class="chart-avg">Schnitt: {{avgHumid}}%</span>
7   </div>
8   <div class="chart-container">
9     <canvas id="luminanceHistoryChart"></canvas>
10  </div>
11 </div>
12
13 <style>
14   .history-card {
15     background: #e3e3e3;
16     border-radius: 16px;
17     padding: 16px 16px 8px 16px;
18     color: white;
19     font-family: 'Segoe UI', sans-serif;
20     height: calc(100% - 32px);
21     display: flex;
22     flex-direction: column;
23   }
24
25   .chart-header {
26     display: flex;
27     justify-content: space-between;
28     align-items: center;
29     margin-bottom: 15px;
30   }
31
32   .chart-title {
```

```

33     font-size: 0.9rem;
34     font-weight: 600;
35     text-transform: uppercase;
36     letter-spacing: 1px;
37     color: #a0aec0;
38 }
39
40 .chart-avg {
41     font-size: 0.85rem;
42     background: rgba(255, 255, 255, 0.1);
43     padding: 2px 8px;
44     border-radius: 4px;
45     color: #a0aec0;
46 }
47
48 .chart-container {
49     flex-grow: 1;
50     position: relative;
51 }
52 </style>
53
54 <script>
55     (function (scope) {
56         let chart;
57
58         function initChart(data) {
59             const ctx = document.getElementById('luminanceHistoryChart').
getContext('2d');
60
61             const gradient = ctx.createLinearGradient(0, 0, 0, 200);
62             gradient.addColorStop(0, 'rgba(27, 213, 237, 0.4)');
63             gradient.addColorStop(1, 'rgba(27, 213, 237, 0)');
64
65             chart = new Chart(ctx, {
66                 type: 'line',
67                 data: {
68                     labels: data.map(d => d.time),
69                     datasets: [{
70                         label: 'Helligkeit',
71                         data: data.map(d => d.value),
72                         borderColor: '#1bd5ed',
73                         borderWidth: 3,
74                         pointRadius: 0,
75                         pointHoverRadius: 5,
76                         fill: true,
77                         backgroundColor: gradient,
78                         tension: 0.4
79                     }]
80                 },
81                 options: {
82                     responsive: true,
83                     maintainAspectRatio: false,
84                     plugins: {
85                         legend: {display: false}
86                     },
87                     layout: {
88                         padding: {

```

```

89         bottom: 10
90     },
91 },
92 scales: {
93     x: {
94         display: true,
95         grid: {
96             display: false,
97             drawBorder: false
98         },
99         ticks: {
100             color: '#718096',
101             font: {size: 10},
102             maxRotation: 0,
103             autoSkip: true,
104             maxTicksLimit: 8
105         }
106     },
107     y: {
108         grid: {
109             color: 'rgba(255,255,255,0.05)',
110             drawBorder: false
111         },
112         ticks: {
113             color: '#718096',
114             font: {size: 10}
115         }
116     }
117 }
118 });
119 }
120 }
121
122 scope.$watch('msg', function (msg) {
123     if (!msg || !msg.payload) return;
124
125     if (Array.isArray(msg.payload)) {
126         const values = msg.payload.map(d => d.value);
127         scope.avgHumid = (values.reduce((a, b) => a + b, 0) / values.
length).toFixed(1);
128
129         if (!chart) {
130             initChart(msg.payload);
131         } else {
132             chart.data.labels = msg.payload.map(d => d.time);
133             chart.data.datasets[0].data = values;
134             chart.update('none');
135         }
136     }
137 });
138 })(scope);
139 </script>

```

## Kapitel 5

# Ganzer Programmcode für ESP32

```
1 from machine import ADC, Pin, SoftI2C
2 from dht import DHT22
3 from time import sleep
4 import json
5 import ssd1306
6 import network
7 from umqtt.simple import MQTTClient
8
9 # Connection pins for the temperature and humidity module
10 RED_LED_PIN = 2
11 ORANGE_LED_PIN = 17
12 GREEN_LED_PIN = 5
13
14 led_red = Pin(RED_LED_PIN, Pin.OUT, drive=Pin.DRIVE_0)
15 led_orange = Pin(ORANGE_LED_PIN, Pin.OUT, drive=Pin.DRIVE_0)
16 led_green = Pin(GREEN_LED_PIN, Pin.OUT, drive=Pin.DRIVE_0)
17
18 dht_device = DHT22(Pin(4, Pin.IN))
19
20 # Connection pin for the luminance module
21 adc_luminance_meter = ADC(Pin(34, Pin.IN))
22
23 # Connection pins for the motion detection module
24 motion_detector = Pin(16, Pin.IN)
25 motion_led = Pin(15, Pin.OUT, drive=Pin.DRIVE_0)
26
27 # Connection pins for the lcd display module (SSD1306)
28 display = ssd1306.SSD1306_I2C(128, 64, SoftI2C(sda=Pin(21), scl=Pin(22)))
29
30 wlan = network.WLAN(network.STA_IF)
31
32 client_id = "ESP_32_MQTT"
33 server = "mosquitto.nodered-fi.ipv64.net"
34 port = 1883
35 user = "FI"
36 password = "FI"
37 mqtt_client = MQTTClient(client_id, server, port, user, password, keepalive=0, ssl
    =False, ssl_params={})
38
39
```

```

40 def mqtt_connect():
41     mqtt_client.connect()
42
43
44 def do_connect():
45     wlan.active(True)
46     if not wlan.isconnected():
47         print('connecting to network...')
48         wlan.connect('FI24-Hotspot', 'BszWsw11#')
49         while not wlan.isconnected():
50             print("WLAN is not connected")
51             pass
52
53     print('network config:', wlan.ifconfig())
54
55
56 def send_mqtt_message(message):
57     channel = "Met/FI/Timmel2007"
58     mqtt_client.publish(channel, message)
59
60
61 def set_temperature_lights(red_on, orange_on, green_on):
62     led_red.value(red_on)
63     led_orange.value(orange_on)
64     led_green.value(green_on)
65
66
67 def handle_temperature_led_lights(temp):
68     if temp >= 25:
69         set_temperature_lights(1, 0, 0)
70     elif temp >= 25 and temp >= 20:
71         set_temperature_lights(0, 1, 0)
72     else:
73         set_temperature_lights(0, 0, 1)
74
75
76 def get_device_temperature_and_humidity():
77     dht_device.measure()
78     temp = dht_device.temperature()
79     humid = dht_device.humidity()
80     return temp, humid
81
82
83 def get_luminance():
84     gamma = 0.7
85     rl10 = 50
86     ref_voltage = 3.3
87     voltage = (adc_luminance_meter.read() / 4) / 1024 * ref_voltage
88     resistance = 2000 * voltage / (1 - voltage / ref_voltage)
89     return pow((rl10 * 1e3) * pow(10, gamma) / resistance, (1 / gamma))
90
91
92 def get_motion():
93     return bool(motion_detector.value())
94
95
96 def display_text(lineOne, lineTwo, lineThree, lineFour, lineSix):

```



```

97     display.fill(0)
98     display.text(lineOne, 0, 0)
99     display.text(lineTwo, 0, 10)
100    display.text(lineThree, 0, 20)
101    display.text(lineFour, 0, 30)
102    display.text(lineSix, 0, 50)
103    display.show()
104
105
106    def set_motion_led(motion):
107        motion_led.value(motion == True)
108
109
110    def handle_lcd_data_display(temperature, humidity, motion, luminance, wlan):
111        display_text(f"Temperatur: {temp}", f"Humidity: {humid}", f"Motion: {motion}",
112                    f"Lux: {luminance}", f"WLAN: {wlan}")
113
114    do_connect()
115    mqtt_connect()
116
117    while True:
118        temp, humid = get_device_temperature_and_humidity()
119        handle_temperature_led_lights(temp)
120        motion = get_motion()
121        luminance = get_luminance()
122        set_motion_led(motion)
123        handle_lcd_data_display(temp, humid, motion, luminance, wlan.isconnected())
124
125        raw_json_object = {
126            "Temperatur": temp,
127            "Luftfeuchtigkeit": humid,
128            "Bewegung": motion,
129            "Helligkeit": luminance
130        }
131
132        json_object = json.dumps(raw_json_object)
133
134        print(json_object)
135
136        send_mqtt_message(json_object)
137        sleep(1)

```